

Dataset 1: patients.csv

```
%%writefile patients.csv
patient_id,patient_name,city,age,gender,blood_group,insurance_status
101,Rahul Sharma,Hyderabad,35,Male,O+,Active
102,Priya Reddy,Bangalore,29,Female,A+,Active
103,Amit Kumar,Mumbai,42,Male,B+,Inactive
104,Sneha Patel,Chennai,31,Female,O+,Active
105,Farhan Ali,Delhi,55,Male,AB+,Active
106,Neha Singh,,38,Female,A+,Inactive
107,Arjun Verma,Pune,26,Male,B+,Active
108,Meera Nair,Kochi,48,Female,O-,Active
109,Kiran Rao,Hyderabad,33,Male,,Inactive
110,Nisha Reddy,Bangalore,41,Female,A+,Active
```

Dataset 2: appointments.csv

```
%%writefile appointments.csv
appointment_id,patient_id,doctor_name,department,appointment_date,consultation_fee
5001,101,Dr. Ramesh,Cardiology,2025-01-10,1500,Completed
5002,102,Dr. Suresh,Neurology,2025-01-11,2000,Completed
5003,101,Dr. Anita,Dermatology,2025-01-15,1000,Completed
5004,103,Dr. Ramesh,Cardiology,2025-01-20,1500,Cancelled
5005,104,Dr. Priya,Orthopedics,2500,2500,Completed
5006,105,Dr. Anita,Dermatology,2025-01-25,1000,Pending
5007,107,Dr. Suresh,Neurology,2025-02-01,2000,Completed
5008,110,Dr. Priya,Orthopedics,2025-02-03,2500,Completed
5009,120,Dr. Ramesh,Cardiology,2025-02-05,1500,Completed
5010,108,Dr. Anita,Dermatology,2025-02-10,,Pending
```

Notice:

```
patient_id 120 does not exist

consultation_fee has null
```

city has null

blood_group has null

Useful for joins and null handling.

Dataset 3: patient_preferences.json

```
%%writefile patient_preferences.json
[
  {
    "patient_id":101,
    "preferred_hospital":"Apollo",
    "contact":{
      "phone":"9876500011",
      "email":"rahul@gmail.com"
    }
  },
  {
    "patient_id":102,
    "preferred_hospital":"Yashoda",
    "contact":{
      "phone":null,
      "email":"priya@gmail.com"
    }
  },
  {
    "patient_id":103,
    "preferred_hospital":"Care",
    "contact":{
      "phone":"9876500013",
      "email":null
    }
  },
  {
    "patient_id":104,
    "preferred_hospital":null,
    "contact":{
      "phone":"9876500014",
      "email":"sneha@gmail.com"
    }
  }
]
```

```
}  
]
```

Part 1 - CSV Ingestion

1. Read `patients.csv`.
 2. Read `appointments.csv`.
 3. Display schema.
 4. Count records.
 5. Display first 5 rows.
 6. Display distinct cities.
 7. Display distinct departments.
 8. Save patients as parquet.
 9. Read parquet file.
 10. Compare CSV and parquet record counts.
-

Part 2 - Filtering

11. Display patients from Hyderabad.
12. Display female patients.
13. Display patients older than 40.
14. Display appointments with status Completed.
15. Display Pending appointments.

16. Display appointments with fee > 1500.
 17. Display patients with Active insurance.
 18. Display patients with Inactive insurance.
 19. Display patients having blood group O+.
 20. Display appointments in Cardiology department.
-

Part 3 - Null Handling

21. Find null cities.
 22. Find null blood groups.
 23. Find null consultation fees.
 24. Count nulls column-wise.
 25. Fill missing city with Unknown.
 26. Fill missing blood group with Not Available.
 27. Fill missing consultation fee with 0.
 28. Drop rows with null consultation fee.
 29. Create data_quality_status column.
 30. Count Complete vs Incomplete records.
-

Part 4 - Built-in Functions

31. Convert patient names to uppercase.
32. Convert patient names to lowercase.

33. Find length of patient names.
 34. Extract first 3 letters of names.
 35. Create age_group column.
 36. Create insurance_flag column.
 37. Create senior_citizen flag.
 38. Concatenate patient name and city.
 39. Trim spaces from names.
 40. Replace city names with uppercase.
-

Part 5 - GroupBy and Aggregations

41. Count patients by city.
 42. Count patients by gender.
 43. Count patients by blood group.
 44. Count appointments by department.
 45. Find average age by city.
 46. Find maximum age by city.
 47. Find minimum age by city.
 48. Find average consultation fee by department.
 49. Find total consultation fee by department.
 50. Find department generating highest revenue.
-

Part 6 - Joins

51. Perform Inner Join.
 52. Perform Left Join.
 53. Perform Right Join.
 54. Perform Full Join.
 55. Find patients with no appointments.
 56. Find appointments with no matching patient.
 57. Count appointments per patient.
 58. Find total fees paid per patient.
 59. Find highest spending patient.
 60. Find patient-wise appointment count.
-

Part 7 - Window Functions

61. Rank patients by total fees paid.
62. Dense rank patients by total fees.
63. Row number based on fees.
64. Find top patient.
65. Find top 3 patients.
66. Find highest spending patient per city.
67. Find lowest spending patient per city.

68. Calculate running total of fees.

69. Use `lead()` on fees.

70. Use `lag()` on fees.

Part 8 - JSON Processing

71. Read `patient_preferences.json`.

72. Display schema.

73. Flatten `contact.phone`.

74. Flatten `contact.email`.

75. Find null phone numbers.

76. Find null emails.

77. Find null preferred hospitals.

78. Fill missing phone values.

79. Fill missing email values.

80. Join preferences with patients.

Part 9 - Spark SQL

81. Create temp view patients.

82. Create temp view appointments.

83. Display all patients using SQL.

84. Display Hyderabad patients using SQL.

85. Count patients by city using SQL.
 86. Count appointments by department using SQL.
 87. Find average consultation fee by department.
 88. Find highest consultation fee.
 89. Find patient-wise appointment count.
 90. Find top 5 spending patients.
-

Part 10 - ETL Project

91. Read CSV files.
92. Read JSON file.
93. Handle null values.
94. Join datasets.
95. Create age_group column.
96. Create revenue metrics.
97. Calculate patient-wise spending.
98. Calculate department revenue.
99. Save final output as parquet.
100. Generate Hospital Analytics Report.